PREDICTIVE ANALYTICS

PROJECT REPORT

(Project Semester- August-December 2026)

(AIRGUARD INDIA: AIR QUALITY ANALYSIS AND POLLUTION ALERT PREDICTION)

Submitted by

Manisha Kumari

Registration No. 12403722

Programme: Computer Science and Engineering

Course Code: INT234

Under the Guidance of

Dr. Mrinalini Rana

U.Id: 22138

Discipline of CSE/IT

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara

DECLARATION

I, Manisha Kumari, student of Computer Science and Engineering (Section NK) under CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 26/10/2026 Signature

Registration No. 12403722 Manisha Kumari

Name of the student

CERTIFICATE

This is to certify that Manisha Kumari bearing Registration no. 12403722 has completed INT234 project titled, **"AirGuard India: Air Quality Analysis and Pollution Alert Prediction"** under my guidance and supervision. To the best of my knowledge, the present work is the result of her original development, effort and study.

Signature and Name of the Supervisor: Dr. Mrinalini Rana
Designation of the Supervisor: Associate Professor
School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab.

Date: 26/10/2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my teacher and project guide, Dr. Mrinalini Rana, for her valuable guidance, clear explanation of the topics and constant encouragement during the INT234 Predictive Analytics course and this project. The practical sessions conducted in the class gave me the base to complete this work.

I am thankful to the Lovely School of Computer Science and Engineering and Lovely Professional University, Phagwara, for giving me the opportunity and the facilities to learn and to work on a real-life data science project.

I also thank the Central Pollution Control Board of India for collecting the air quality data, and the Kaggle community for making the dataset available to students. I am grateful to the developers of Python, pandas, matplotlib, seaborn and scikit-learn, whose free tools made this analysis possible.

Finally, I thank my family and my friends for their support and motivation.

Manisha Kumari

Registration No. 12403722

TABLE OF CONTENT

1. Introduction **8**
1.1 Background 8
1.2 Problem Statement 8
1.3 Objectives 8
1.4 Scope of the Project 9
1.5 Tools and Libraries Used 9
1.6 Organisation of the Report 9
2. Source of Dataset **10**
2.1 Origin of the Dataset 10
2.2 Description of the Columns 10
2.3 AQI Categories 11
2.4 Summary Statistics 11
2.5 Limitations of the Dataset 11
3. EDA Process **13**
3.1 Loading the Data and First Look 13
3.2 Missing Value Analysis 13
3.3 Data Cleaning Steps 13
3.4 Creating the ALERT and Month Columns 14
3.5 Distribution of AQI 14
3.6 Correlation between Pollutants and AQI 15
3.7 City-wise Analysis 16
3.8 AQI Category Counts 18
3.9 PM2.5 versus AQI 18
3.10 Seasonal Pattern 19
3.11 Normalization and Train-Test Split 20
4. Analysis on Dataset **21**
4.1 Simple Linear Regression 21
4.2 Multiple Linear Regression 23
4.3 Logistic Regression 25
4.4 K-Nearest Neighbours (KNN) 26
4.5 Gaussian Naive Bayes 28
4.6 Decision Tree 30
4.7 Support Vector Machine (SVM) 31
4.8 Comparison of the Classification Models 33
5. Conclusion **36**
6. Future Scope **37**
7. References **38**

LIST OF FIGURES

Figure 1: Percentage of missing values in each column 13
Figure 2: Distribution of AQI values 15
Figure 3: Correlation heatmap of pollutants and AQI 16
Figure 4: Average AQI of each city 17
Figure 5: Percentage of alert days in each city 17
Figure 6: Number of days in each AQI category 18
Figure 7: Scatter plot of PM2.5 and AQI 19
Figure 8: Average AQI in each month of the year 20
Figure 9: Simple Linear Regression line (PM2.5 and AQI) 22
Figure 10: Actual AQI versus predicted AQI (Multiple Linear Regression) 25
Figure 11: Confusion matrix of Logistic Regression 26
Figure 12: Accuracy of KNN for different values of K 28
Figure 13: Confusion matrix of KNN 28
Figure 14: Confusion matrix of Gaussian Naive Bayes 29
Figure 15: Decision Tree of depth 3 for predicting alert days 31
Figure 16: Confusion matrix of the Decision Tree 31
Figure 17: Confusion matrix of the SVM 33
Figure 18: ROC curve of the SVM model 33
Figure 19: Comparison of the classification models 35

LIST OF TABLES

Table 1: Python libraries used in the project 9
Table 2: Columns of the dataset city_day.csv 10
Table 3: AQI categories of CPCB and the alert label used in the project 11
Table 4: Summary statistics of selected columns (original data) 11
Table 5: Data cleaning steps and their effect 14
Table 6: Five cities with the highest average AQI and their alert percentage 18
Table 7: Result of Simple Linear Regression (test data) 22
Table 8: Comparison of the two regression models (test data) 23
Table 9: Coefficients of the Multiple Linear Regression model (normalized inputs) 23
Table 10: Accuracy of KNN for different values of K 27
Table 11: Comparison of the classification models (test data, sorted by recall) 34
Table 12: Confusion matrix values of all classification models 34

1. Introduction

1.1 Background

Air pollution is one of the most serious environmental and public-health problems in India. Many Indian cities regularly record pollutant levels far above the limits that are considered safe for human health. Fine particles such as PM2.5 and PM10 can enter the lungs and the bloodstream, and gases such as nitrogen dioxide (NO2), sulphur dioxide (SO2) and carbon monoxide (CO) add to the damage. To make this information easy to understand for common people, the Central Pollution Control Board (CPCB) of India converts the readings of several pollutants into a single number called the Air Quality Index (AQI). A low AQI means clean air and a high AQI means that the air is harmful.

Every day, monitoring stations in different cities record the concentration of many pollutants. Over the years this has produced a large amount of data. Data science and machine learning give us tools to study this data, to find which pollutants are linked with a high AQI, to compare cities with each other, and to build models that can estimate the AQI and warn about days on which the air quality will be poor.

1.2 Problem Statement

The aim of this project is to study the daily air quality data of Indian cities and to answer two practical questions. First, can the numerical value of the AQI be predicted from the readings of the pollutants? This is a regression problem. Second, can a day be classified as an alert day (Poor, Very Poor or Severe air quality) or a normal day using only the pollutant readings? This is a binary classification problem. The project is named AirGuard India because the final aim is to guard people by giving an early warning about days with harmful air.

1.3 Objectives

- To study the AQI levels of different Indian cities and compare them.
- To check the relationship between the pollutants and the AQI using correlation and graphs.
- To clean the dataset by handling missing values and unwanted rows.
- To build and compare regression models (Simple Linear Regression and Multiple Linear Regression) for predicting the AQI.
- To build and compare classification models (Logistic Regression, KNN, Gaussian Naive Bayes, Decision Tree and SVM) for predicting alert days.

- To evaluate every model with standard metrics and to explain the results in a simple way.

1.4 Scope of the Project

The project is limited to the topics studied in the INT234 Predictive Analytics course. It uses the historical city-wise daily air quality dataset, performs data cleaning and exploratory data analysis (EDA), applies Min-Max normalization, and then applies the regression and classification techniques that were taught in the class. All the work is done in Python using the Spyder environment. The project is a learning prototype built on historical data. It does not use live sensor data and it is not an official warning system.

1.5 Tools and Libraries Used

The complete program is written in Python. The libraries used in the program and the purpose of each library are given in Table 1.

Table 1: Python libraries used in the project

Library	Purpose
pandas	Reading the CSV file, cleaning the data, grouping and summarising
numpy	Numerical calculations such as the square root used for RMSE
matplotlib	Drawing histograms, scatter plots, bar charts, ROC curve and the tree
seaborn	Heatmap of correlation, count plot and scatter plot
scikit-learn	Scaling, train-test split, all models and all evaluation metrics

1.6 Organisation of the Report

Chapter 2 describes the source and structure of the dataset. Chapter 3 explains the EDA process, including cleaning, graphs and preparation of the data. Chapter 4 contains the analysis of the dataset. Every analysis in this chapter is written in five parts: introduction, general description, specific requirements with functions and formulas, analysis results, and visualization. Chapter 5 gives the conclusion, Chapter 6 gives the future scope and Chapter 7 lists the references.

GitHub Repo Link: https://github.com/ananyaofficial2025-prog/Manisha_Airguard_India

LinkedIn post: https://lnkd.in/p/dYEy74C4

Dataset:https://www.kaggle.com/datasets/saikiranudayana/india-air-quality-index-aqidataset-20232025

2. Source of Dataset

2.1 Origin of the Dataset

The dataset used in this project is the file city_day.csv from the publicly available data set "Air Quality Data in India (2015 - 2020)", which is hosted on the Kaggle website. The data in this data set was collected from the Central Pollution Control Board (CPCB) of India, which operates the air quality monitoring network of the country. The file contains the city-wise daily average of the pollutant concentrations together with the AQI and the AQI category. In this project the file is read from the local computer; no internet connection or live data is needed.

The file has 29,531 rows and 16 columns. Each row is the record of one city on one day. The data covers 26 Indian cities and the period from 2015-01-01 to 2020-07-01.

2.2 Description of the Columns

Table 2 explains the columns of the dataset. The units of the pollutant columns are micrograms per cubic metre (µg/m³), except CO which is in milligrams per cubic metre (mg/m³).

Table 2: Columns of the dataset city_day.csv

Column	Type	Meaning
City	Text	Name of the city where the reading was taken
Date	Date	Date of the reading (day-month-year)
PM2.5	Number	Fine particles smaller than 2.5 micrometres
PM10	Number	Particles smaller than 10 micrometres
NO, NO2, NOx	Number	Nitrogen monoxide, nitrogen dioxide and total nitrogen oxides
NH3	Number	Ammonia
CO	Number	Carbon monoxide
SO2	Number	Sulphur dioxide
O3	Number	Ozone
Benzene, Toluene, Xylene	Number	Volatile organic compounds
AQI	Number	Air Quality Index value of the day (target for regression)
AQI_Bucket	Text	Category of the AQI such as Good, Poor or Severe

2.3 AQI Categories

The CPCB divides the AQI into six categories. These categories decide the label that is used in the classification part of this project. A day is called an alert day when the air quality is Poor, Very Poor or Severe, which means the AQI is greater than 200.

Table 3: AQI categories of CPCB and the alert label used in the project

AQI range	Category	ALERT label
0 - 50	Good	0 (No Alert)
51 - 100	Satisfactory	0 (No Alert)
101 - 200	Moderate	0 (No Alert)
201 - 300	Poor	1 (Alert)
301 - 400	Very Poor	1 (Alert)
401 and above	Severe	1 (Alert)

2.4 Summary Statistics

Table 4 shows the summary statistics of some important columns, taken with the describe() function of pandas on the original file. The AQI column has a mean of 166.46 but a median of only 118.00, and the maximum value is 2049.00. This shows that the AQI is skewed to the right and that a few extremely polluted days pull the average up. The same is true for PM2.5, PM10 and CO.

Table 4: Summary statistics of selected columns (original data)

Statistic	PM2.5	PM10	NO2	CO	SO2	O3	AQI
count	24,933	18,391	25,946	27,472	25,677	25,509	24,850
mean	67.45	118.13	28.56	2.25	14.53	34.49	166.46
std	64.66	90.61	24.47	6.96	18.13	21.69	140.70
min	0.04	0.01	0.01	0.00	0.01	0.01	13.00
50%	48.57	95.68	21.69	0.89	9.16	30.84	118.00
max	949.99	1000.00	362.21	175.81	193.86	257.73	2049.00

2.5 Limitations of the Dataset

- Many readings are missing, especially for Xylene, PM10, NH3 and Toluene (see Chapter 3).

- The AQI is missing for about 16 percent of the rows, so these rows cannot be used for supervised learning.
- Some AQI values are extremely high (the maximum is above 2000), although the official scale ends at 500. These extreme days are kept in the analysis because they are part of the original data, but they increase the prediction errors of the regression models.
- Only daily city averages are available, so differences inside a city are not captured.

3. EDA Process

Exploratory Data Analysis (EDA) means studying the data with the help of summaries and graphs before building any model. In this project the EDA was done in the same order that was followed in the class: load the data, look at its structure, identify the missing values, clean the data, and then draw graphs to understand the patterns.

3.1 Loading the Data and First Look

The data was loaded with the pandas function read_csv(). The functions head(), info() and describe() were used to see the first rows, the data types and the summary statistics. The function isna().sum() was used to count the missing values of every column.

data = pd.read_csv("city_day.csv") print(data.info()) print(data.isna().sum())

3.2 Missing Value Analysis

The percentage of missing values in each column is shown in Figure 1. Xylene has the highest share of missing values (about 61 percent), followed by PM10 (about 38 percent), NH3 (35 percent) and Toluene (27 percent). The target column AQI and the column AQI_Bucket are each missing in about 16 percent of the rows.

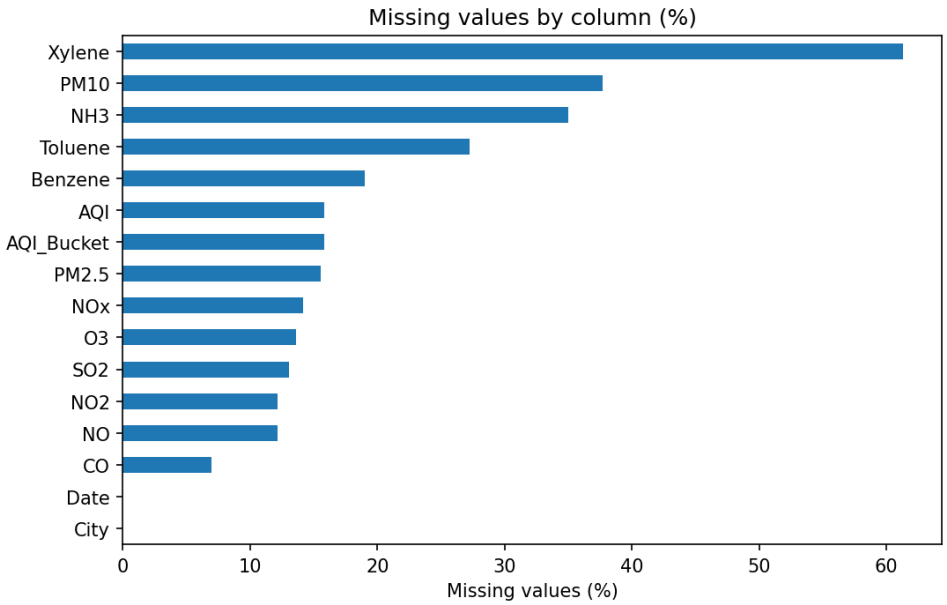


Figure 1: Percentage of missing values in each column

3.3 Data Cleaning Steps

The cleaning was done in the following order, and the effect of each step is given in Table 5.

Table 5: Data cleaning steps and their effect

Step	Method used	Effect
Duplicate rows	drop_duplicates()	0 duplicate rows were found
Missing AQI	dropna(subset=["AQI"])	Rows reduced from 29,531 to 24,850
Columns with more than 40% missing	Loop with isna().mean()	Xylene (61.3%) was dropped
Missing pollutant values	fillna() with column mean	No missing values remained in the 11 pollutant columns
Date column	pd.to_datetime()	Month was extracted from Date
Alert label	AQI > 200	New column ALERT (0 or 1) created

Rows without an AQI value were removed because the AQI is the target of the regression and is also used to create the alert label; such rows cannot be used to train or test a model. The pollutant columns that had more than 40 percent missing values were removed because filling so many values with a single average would add more guesswork than real information. The remaining missing values of the numeric pollutant columns were replaced by the mean of that column, which is the method taught in the class for numeric columns.

3.4 Creating the ALERT and Month Columns

The target for classification was created with the statement data["ALERT"] = (data["AQI"] > 200).astype(int). Out of 24,850 cleaned rows, 6,456 rows (25.98 percent) are alert days and 18,394 rows are normal days. So about one day out of every four is an alert day. The dataset is therefore moderately imbalanced, and for this reason recall and F1-score are studied together with accuracy in the classification part. The column Month (1 to 12) was extracted from the Date column to see whether the season has any effect.

3.5 Distribution of AQI

The histogram in Figure 2 shows how the AQI values are spread. Most of the days have an AQI between 50 and 250, and the number of days falls slowly as the AQI becomes higher. The long tail on the right side shows the small number of extremely polluted days. The shape is right-skewed, which agrees with the summary statistics.

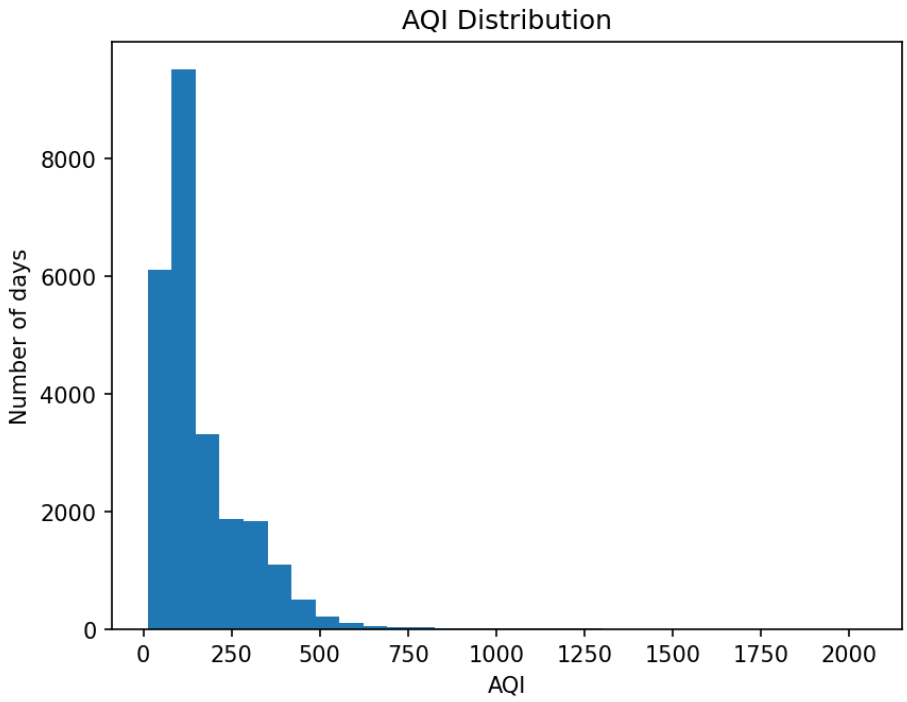


Figure 2: Distribution of AQI values

3.6 Correlation between Pollutants and AQI

A correlation heatmap was drawn with seaborn to see how strongly each pollutant is related to the AQI (Figure 3). The correlation values with AQI are: CO 0.68, PM2.5 0.65, NO2 0.54, PM10 0.50, SO2 0.48, NOx 0.47 and NO 0.45. Toluene (0.26), O3 (0.19), NH3 (0.14) and Benzene (0.04) show weak correlation. A correlation near +1 means that the two columns increase together. The strong positive values of CO and PM2.5 tell us that these two pollutants are the most useful inputs for predicting the AQI. The heatmap also shows that NO, NO2 and NOx are strongly correlated with each other, which is expected because they are chemically related.

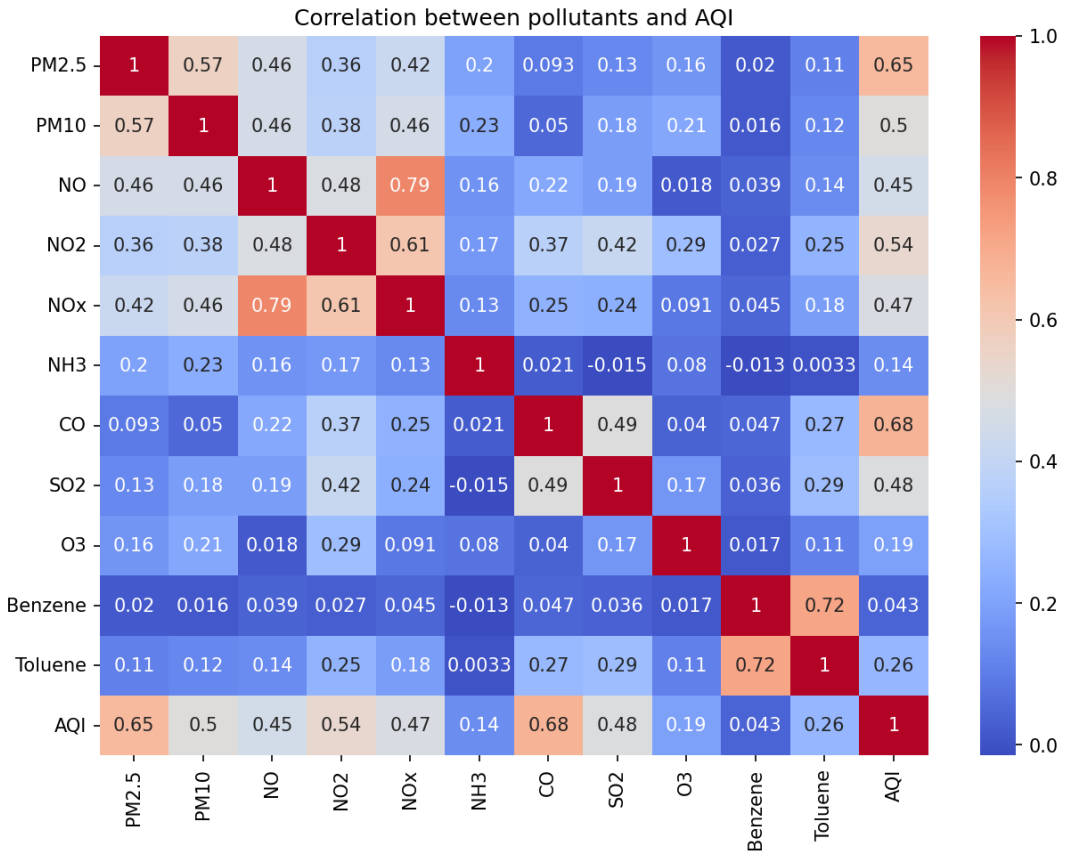


Figure 3: Correlation heatmap of pollutants and AQI

3.7 City-wise Analysis

The average AQI of each city is shown in Figure 4. Ahmedabad has the highest average AQI (452.12), followed by Delhi (259.49), Patna (240.78), Gurugram (225.12) and Lucknow (217.97). The cleanest cities in the data are Aizawl (34.77), Shillong (53.80) and Coimbatore (73.02).

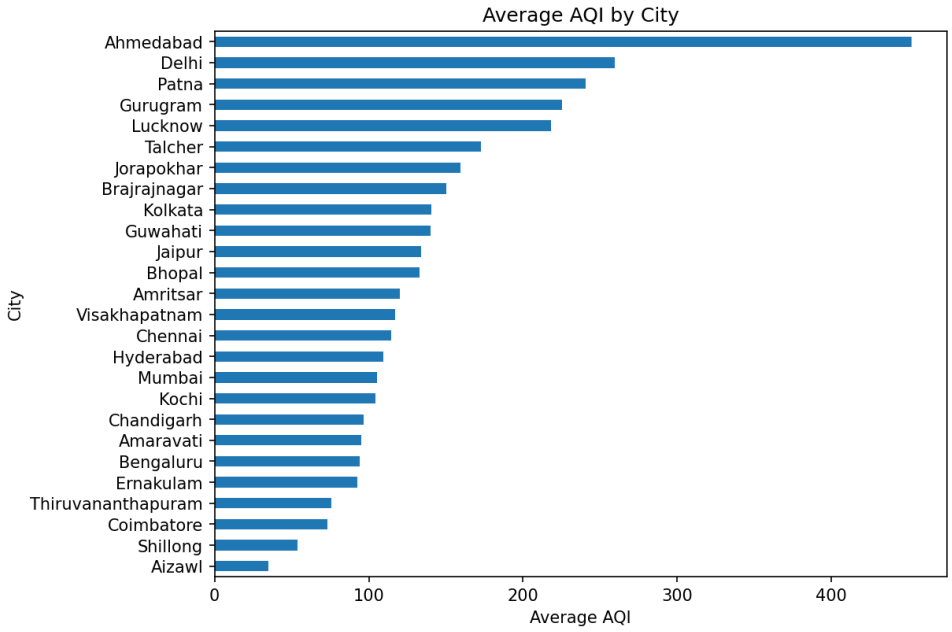


Figure 4: Average AQI of each city

Figure 5 shows the percentage of days on which each city had an alert. This graph gives the same message in a different form: in the most polluted cities more than half of the recorded days were Poor, Very Poor or Severe, while in the cleanest cities there were almost no alert days. The top five cities are listed in Table 6.

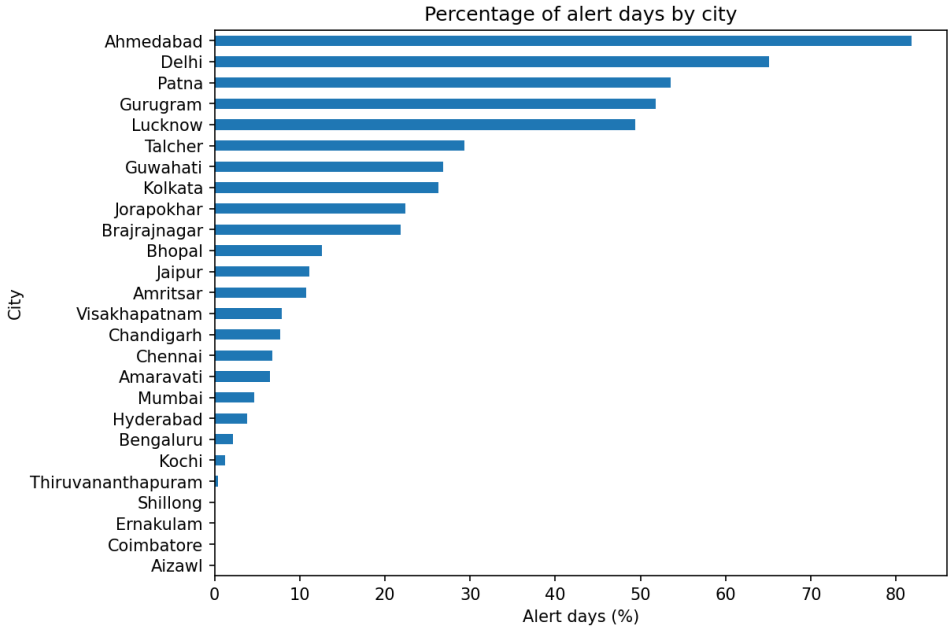


Figure 5: Percentage of alert days in each city

Table 6: Five cities with the highest average AQI and their alert percentage

City	Average AQI	Alert days (%)
Ahmedabad	452.12	81.86
Delhi	259.49	65.08
Patna	240.78	53.53
Gurugram	225.12	51.82
Lucknow	217.97	49.39

3.8 AQI Category Counts

Figure 6 is a count plot of the AQI categories. The largest groups are Moderate (8,829 days) and Satisfactory (8,224 days). Poor has 2,781 days, Very Poor has 2,337 days, Good has 1,341 days and Severe has 1,338 days. Together, the last three alert categories other than Good form the 25.98 percent alert share.

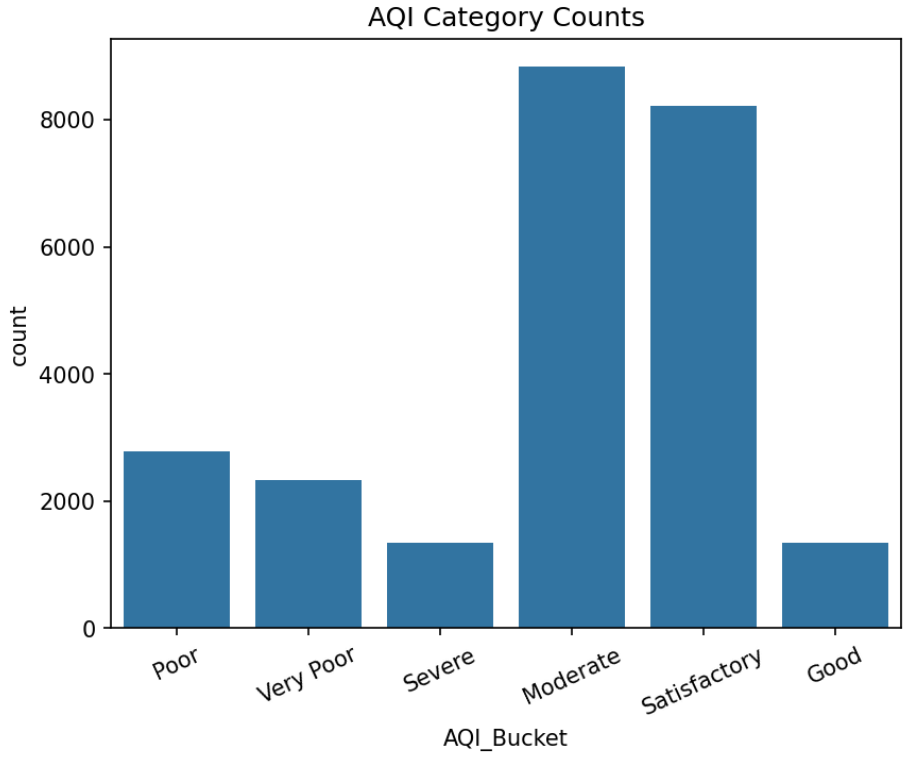


Figure 6: Number of days in each AQI category

3.9 PM2.5 versus AQI

The scatter plot in Figure 7 shows that when PM2.5 increases, the AQI also increases. The points form an upward pattern, but they are not on a perfect straight line. Some points with a low PM2.5 still have a high AQI because the AQI is decided by the worst pollutant of the day

and not by PM2.5 alone. This is the reason why the Simple Linear Regression in Chapter 4 gives only a moderate accuracy and the Multiple Linear Regression, which uses all pollutants, performs much better.

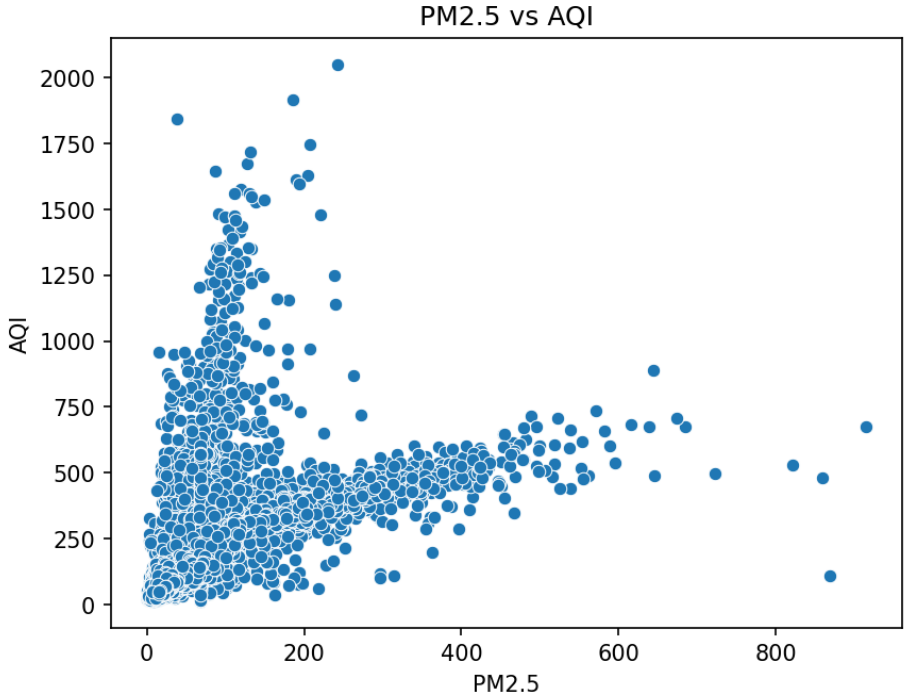


Figure 7: Scatter plot of PM2.5 and AQI

3.10 Seasonal Pattern

Figure 8 shows the average AQI for each month of the year. The air quality is worst in the winter months: November (241.68), January (231.67) and December (227.08). It is best in the rainy months of July (111.85) and August (113.61). In winter the air is cold and still, which traps pollutants near the ground, and the burning of crop residue and the use of heaters add more pollution. In the monsoon the rain washes the pollutants out of the air.

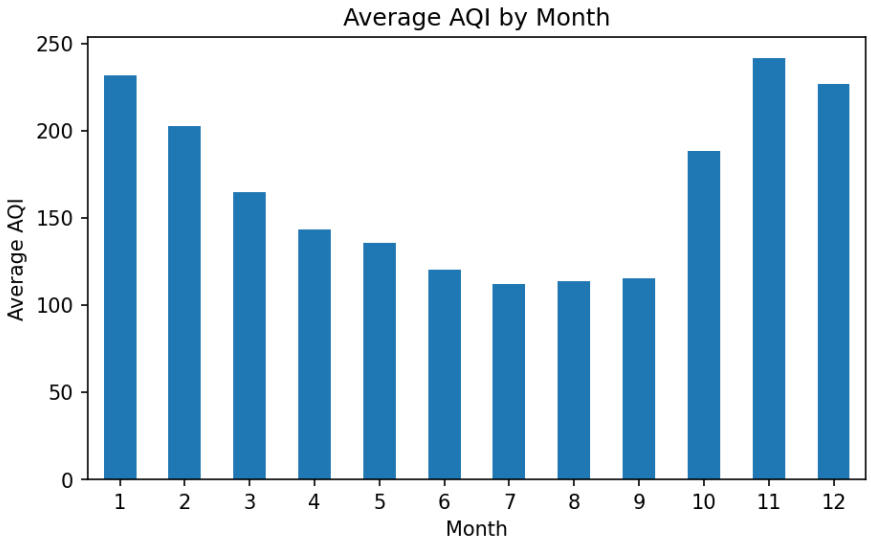


Figure 8: Average AQI in each month of the year

3.11 Normalization and Train-Test Split

The pollutant columns have very different ranges. For example, the CO values are small while the PM10 values are large. Therefore the pollutant columns were normalized with the Min-Max method, so that every pollutant lies between 0 and 1. This is done with MinMaxScaler of scikit-learn as taught in the class.

$$x' = (x - min(x)) / (max(x) - min(x))$$

After this the data was divided into training and testing parts with train_test_split(), using test_size = 0.2 and random_state = 42. So 19,880 rows (80 percent) are used for training and 4,970 rows (20 percent) are used for testing. The same split and the same random_state are used for all the classification models so that they can be compared fairly. The regression models use the target AQI, and the classification models use the target ALERT. The AQI column itself is never used as an input for the classification models, because the ALERT label is created from the AQI and using it would give the answer away.

4. Analysis on Dataset

This chapter contains seven analyses: two regression models for predicting the AQI and five classification models for predicting the alert days. Each analysis is explained under the headings (i) Introduction, (ii) General Description, (iii) Specific Requirements, functions and formulas, (iv) Analysis results and (v) Visualization. The last section of the chapter compares the classification models.

4.1 Simple Linear Regression

i. Introduction

Simple Linear Regression is the most basic prediction technique. It tries to explain one numerical output with the help of only one input. In this project the output is the AQI and the input is PM2.5, because PM2.5 is one of the pollutants with the strongest correlation with the AQI.

ii. General Description

The model draws the best straight line through the points of the scatter plot. The line is chosen in such a way that the total of the squared distances between the actual points and the line is as small as possible. This method is called the method of least squares. After the line is found, the AQI for a new day can be predicted by putting the PM2.5 value of that day in the equation of the line.

iii. Specific Requirements, Functions and Formulas

The input X is the normalized PM2.5 column (a DataFrame with double brackets) and the output y is the AQI column. The functions used are train_test_split(), LinearRegression(), fit(), predict() and the metrics functions mean_absolute_error(), mean_squared_error() and r2_score(). The formulas are:

$$AQI = b0 + b1 \times PM2.5$$

$$MAE = (1/n)\,\Sigma\,|y - \hat{y}| \qquad MSE = (1/n)\,\Sigma\,(y - \hat{y})^2 \qquad RMSE = \sqrt{MSE}$$

$$R^2 = 1 - [\,\Sigma\,(y - \hat{y})^2 / \Sigma\,(y - \bar{y})^2\,]$$

Here y is the actual AQI, $\hat{y}$ is the predicted AQI, $\bar{y}$ is the mean AQI and n is the number of test rows. MAE is the average size of the error, RMSE gives more weight to big errors, and R^2 tells what fraction of the variation of the AQI is explained by the model.

iv. Analysis Results

The fitted line is AQI = 64.65 + 1386.93 × PM2.5 (normalized). The result on the test data is given in Table 7. The R^2 score is 0.4180, which means that PM2.5 alone explains about 42 percent of the variation in the AQI. The MAE of 50.28 means that on an average the prediction is wrong by about 50 AQI points. This is a moderate result, which is expected because the AQI depends on many pollutants.

Table 7: Result of Simple Linear Regression (test data)

Metric	Value
MAE	50.28
MSE	10657.17
RMSE	103.23
R^2 Score	0.4180

v. Visualization

Figure 9 shows the test points in blue and the fitted regression line in red. The line follows the general upward trend of the points, but many points are far away from the line, especially for the high values of PM2.5.

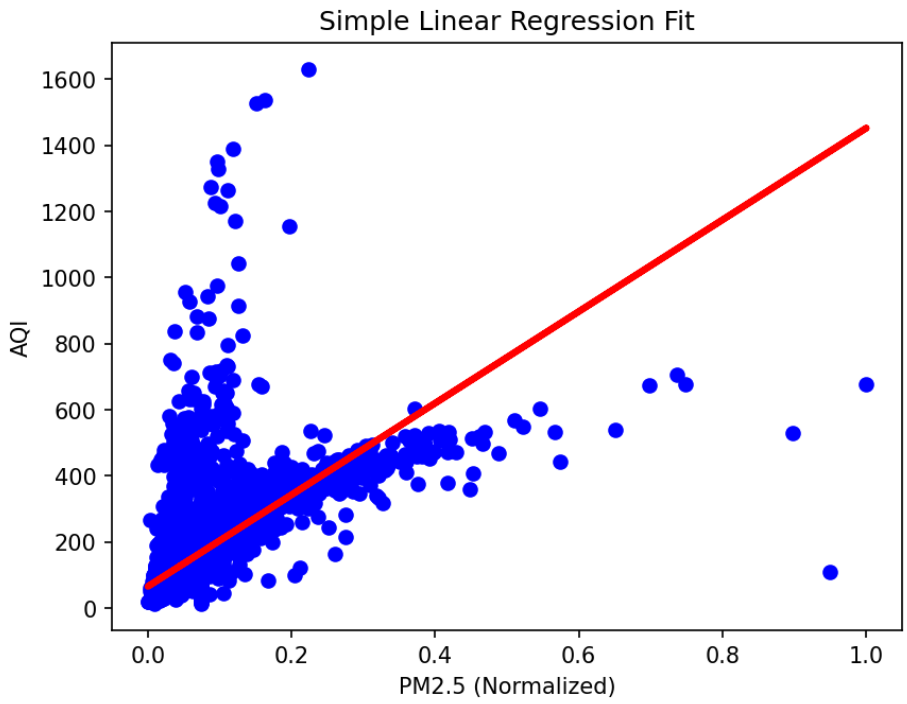


Figure 9: Simple Linear Regression line (PM2.5 and AQI)

4.2 Multiple Linear Regression

i. Introduction

Multiple Linear Regression extends the simple model by using many inputs together. Since the AQI of a day is decided by several pollutants, a model that uses all of them is expected to give better predictions than a model that uses only PM2.5.

ii. General Description

The model finds one coefficient for every input column. A positive coefficient means that the AQI increases when that input increases, and a negative coefficient means the opposite. All the inputs are normalized, so the sizes of the coefficients can be compared with each other in a rough way.

iii. Specific Requirements, Functions and Formulas

The inputs are the 11 pollutant columns (PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene and Toluene) and the Month column. The output is the AQI. The same functions as in the simple model are used. The equation of the model is:

$$AQI = b0 + b1 \cdot PM2.5 + b2 \cdot PM10 + b3 \cdot NO + \ldots + b11 \cdot Toluene + b12 \cdot Month$$

iv. Analysis Results

The model gives an R^2 score of 0.8091, which is much higher than 0.4180 of the simple model. The MAE came down from 50.28 to 31.23 and the RMSE came down from 103.23 to 59.12. So using all pollutants explains about 81 percent of the variation of the AQI. The comparison of the two regression models is given in Table 8 and the coefficients are given in Table 9.

Table 8: Comparison of the two regression models (test data)

Model	MAE	MSE	RMSE	R^2 Score
Simple Linear Regression	50.28	10657.17	103.23	0.4180
Multiple Linear Regression	31.23	3495.38	59.12	0.8091

Table 9: Coefficients of the Multiple Linear Regression model (normalized inputs)

Input	Coefficient
Intercept (b0)	8.29
PM2.5	1024.55
PM10	267.69
NO	-52.72

Input	Coefficient
NO2	88.76
NOx	37.52
NH3	-23.09
CO	2060.75
SO2	126.77
O3	49.65
Benzene	10.65
Toluene	-14.73
Month	-0.27

The coefficients show that CO and PM2.5 have the largest positive effect on the AQI, followed by PM10, SO2 and NO2. The coefficient of Month is almost zero, which means that once the pollutant readings are known, the month adds almost no extra information. The negative coefficients of NO, NH3 and Toluene should not be read as "these pollutants reduce the AQI". They appear because these columns are correlated with other pollutants in the model, and the model adjusts them to get the best overall fit.

v. Visualization

Figure 10 plots the actual AQI against the predicted AQI for the test data. If the model were perfect, all the points would lie on a straight diagonal line. The points are close to a diagonal pattern for the low and middle values, but for very high AQI values the model underestimates or overestimates a lot. These extreme days are the main reason for the large RMSE.

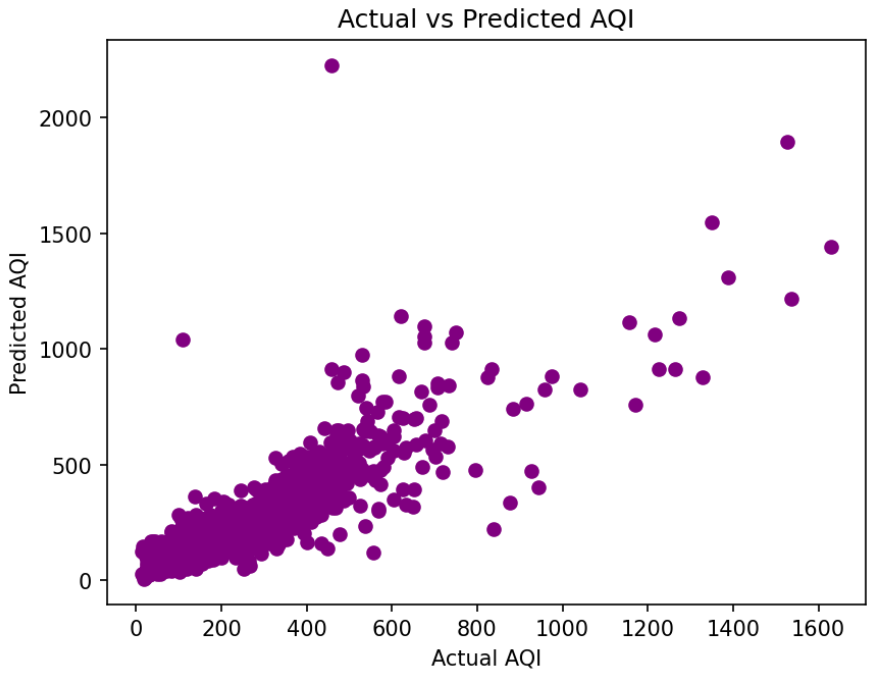


Figure 10: Actual AQI versus predicted AQI (Multiple Linear Regression)

4.3 Logistic Regression

i. Introduction

Logistic Regression is a classification technique, even though the word regression is in its name. It is used when the output has only two possible values. Here the output is the ALERT label: 1 for an alert day and 0 for a normal day.

ii. General Description

The model first calculates a weighted sum of the inputs, as in linear regression, and then passes this sum through the sigmoid function. The sigmoid function converts any number into a value between 0 and 1, which can be read as the probability of an alert. If the probability is 0.5 or more the day is classified as an alert day, otherwise it is classified as a normal day.

iii. Specific Requirements, Functions and Formulas

The inputs are the 11 normalized pollutants and Month. The output is ALERT. The functions used are LogisticRegression(max_iter = 1000), fit(), predict(), confusion_matrix() and classification_report(). The formulas are:

$$z = b0 + b1 \cdot x1 + b2 \cdot x2 + \ldots + bn \cdot xn \qquad P(alert) = 1 / (1 + e^{(-z)})$$

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \qquad Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN) \qquad F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

Here TP is the number of alert days predicted correctly, TN is the number of normal days predicted correctly, FP is the number of normal days wrongly predicted as alert and FN is the number of alert days that the model missed.

iv. Analysis Results

Logistic Regression gave an accuracy of 92.41%, a precision of 92.77%, a recall of 76.71% and an F1-score of 83.98%. In the confusion matrix, 3605 normal days and 988 alert days were predicted correctly, 77 normal days were wrongly flagged as alert and 300 alert days were missed. The precision is very high, which means that when the model raises an alert it is almost always right. But the recall is only about 77 percent, so nearly one alert day out of four is missed. For a warning system this is the weak point of this model.

v. Visualization

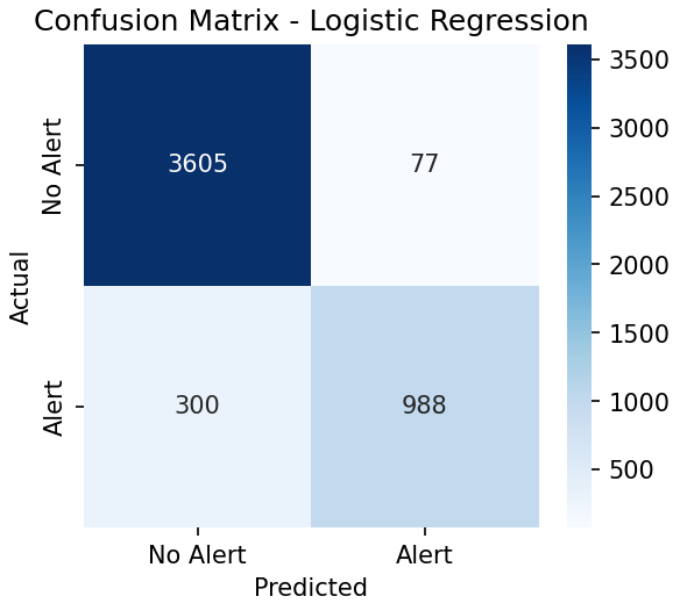


Figure 11: Confusion matrix of Logistic Regression

4.4 K-Nearest Neighbours (KNN)

i. Introduction

K-Nearest Neighbours is a simple classification method that takes a decision by looking at similar examples. It does not build any equation; it only remembers the training data.

ii. General Description

To classify a new day, KNN finds the K training days whose pollutant readings are closest to the new day, and then takes a majority vote of their labels. If most of the K neighbours were alert days, the new day is also classified as an alert day. The value of K must be chosen

carefully: a very small K is sensitive to noise, and a very large K ignores local patterns. Because KNN depends on distance, normalization of the inputs, which was done with MinMaxScaler, is essential.

iii. Specific Requirements, Functions and Formulas

The functions used are KNeighborsClassifier(n_neighbors = K), fit(), predict(), accuracy_score() and confusion_matrix(). A for loop was used to try the odd values of K from 3 to 15 and to select the K with the highest accuracy. The distance between two days is the Euclidean distance:

$$d(a, b) = \sqrt{[\ (a1 - b1)^2 + (a2 - b2)^2 + \ldots + (an - bn)^2\ ]}$$

iv. Analysis Results

Table 10: Accuracy of KNN for different values of K

K	Accuracy
3	93.84%
5	93.74%
7	93.58%
9	93.52%
11	93.38%
13	93.12%
15	93.08%

The best accuracy was obtained for K = 3 (93.84%), and the accuracy decreases slowly as K grows (Table 10 and Figure 12). With K = 3 the KNN model gave an accuracy of 93.84%, a precision of 90.11%, a recall of 85.64% and an F1-score of 87.82%. It correctly found 1103 of the 1288 alert days and missed 185. Among all five classification models KNN has the highest recall, which is very important for an alert system. It should be noted that K was selected using the accuracy on the test data, so the reported KNN score may be slightly optimistic.

v. Visualization

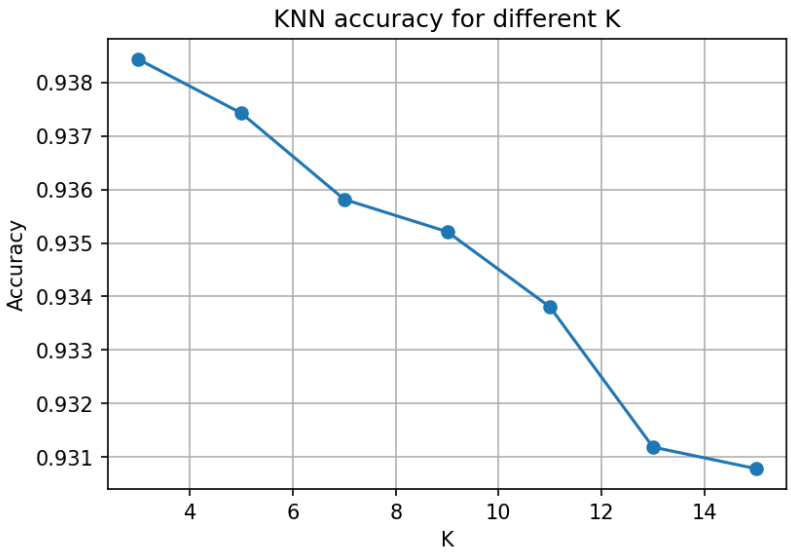


Figure 12: Accuracy of KNN for different values of K

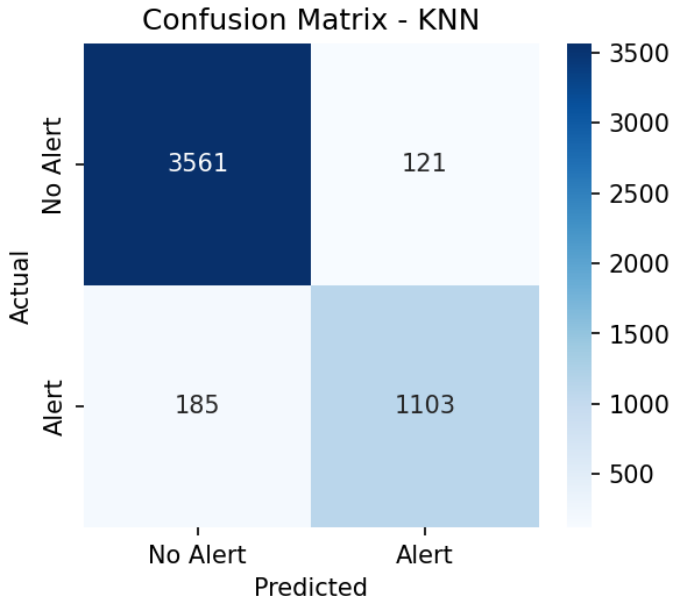


Figure 13: Confusion matrix of KNN

4.5 Gaussian Naive Bayes

i. Introduction

Naive Bayes is a probability-based classification technique that uses Bayes theorem. The word naive is used because it assumes that all input columns are independent of each other, which is rarely true in real data, but the method still works well in many problems.

ii. General Description

For every class (alert and no alert) the model learns the average and the spread of every pollutant. For a new day it calculates how likely the observed readings are under each class, multiplies this with the proportion of the class in the training data, and selects the class with the higher result. The Gaussian version is used here because all the inputs are continuous numbers and are assumed to follow a normal curve inside each class.

iii. Specific Requirements, Functions and Formulas

The functions used are GaussianNB(), fit(), predict(), confusion_matrix() and classification_report(). The formulas are:

$$P(C \mid X) \propto P(C) \times P(x1 \mid C) \times P(x2 \mid C) \times \ldots \times P(xn \mid C)$$

$$P(xi \mid C) = [\ 1 / \sqrt{(2\pi\ \sigma^2)}\ ] \times e^{(-(xi - \mu)^2 / (2\sigma^2))}$$

iv. Analysis Results

Gaussian Naive Bayes gave an accuracy of 90.18%, a precision of 82.73%, a recall of 78.49% and an F1-score of 80.56%. This is the lowest accuracy and F1-score among the five models. The reason is the independence assumption: the pollutants such as NO, NO2 and NOx and also PM2.5 and PM10 are strongly correlated (see the heatmap in Figure 3), so the model counts the same information more than once. The confusion matrix shows 211 false alerts and 277 missed alerts.

v. Visualization

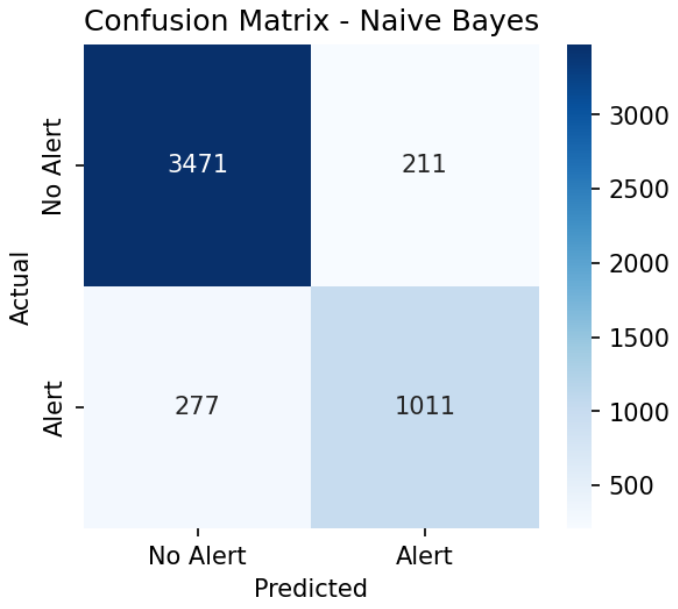


Figure 14: Confusion matrix of Gaussian Naive Bayes

4.6 Decision Tree

i. Introduction

A Decision Tree classifies data by asking a series of yes-or-no questions about the input columns, like a flow chart. It is easy to understand and explain, which is a big advantage when the result has to be explained to non-technical people.

ii. General Description

The tree starts with all the training days at the top (the root). At every step it chooses the pollutant and the cut-off value that separate the alert days from the normal days in the best possible way, and splits the data into two branches. This is repeated until the maximum depth is reached. The final boxes (leaves) give the predicted class. In this project the entropy criterion was used and the depth of the tree was limited to 3, so that the tree remains simple and does not memorise the training data.

iii. Specific Requirements, Functions and Formulas

The functions used are DecisionTreeClassifier(criterion = "entropy", max_depth = 3, random_state = 42), fit(), predict() and plot_tree(). The quality of a split is measured by entropy and information gain:

$$Entropy = - \Sigma\, pi \times log2(pi)$$

$$Information\ Gain = Entropy(parent) - \Sigma\, [\ (ni / n) \times Entropy(child\ i)\ ]$$

An entropy of 0 means that a box contains only one class (a pure box), and an entropy of 1 means that the two classes are mixed equally.

iv. Analysis Results

The Decision Tree gave the highest accuracy of all models: 94.02%, with a precision of 91.67%, a recall of 84.63% and the highest F1-score of 88.01%. It produced 99 false alerts and 198 missed alerts. Although the tree is very small, it works very well. The tree uses only three inputs: PM2.5 (importance 0.80), CO (0.18) and NO (0.02). The root question of the tree is whether the normalized PM2.5 is less than or equal to about 0.098. This agrees with the correlation analysis, where PM2.5 and CO had the strongest relation with the AQI.

v. Visualization

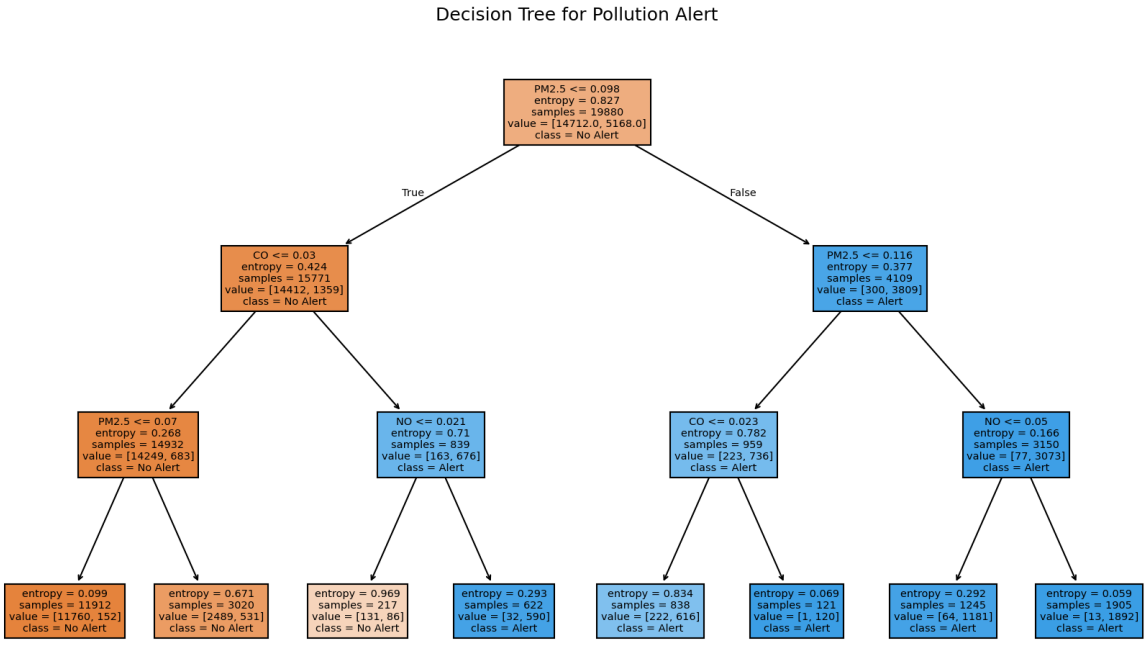


Figure 15: Decision Tree of depth 3 for predicting alert days

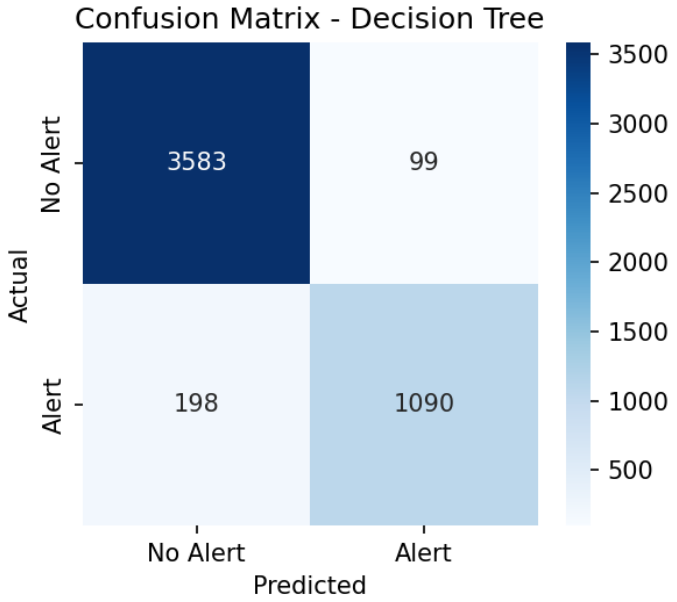


Figure 16: Confusion matrix of the Decision Tree

4.7 Support Vector Machine (SVM)

i. Introduction

A Support Vector Machine is a classification technique that tries to separate the two classes with the best possible boundary. It is one of the strongest classical machine learning methods.

ii. General Description

In the linear form of SVM used here, the model finds a straight boundary (a hyperplane) that separates alert days from normal days with the widest possible gap, which is called the margin. The training points that lie closest to the boundary are called support vectors, and only these points decide the position of the boundary. A wide margin helps the model to work well on new data.

iii. Specific Requirements, Functions and Formulas

The functions used are SVC(kernel = "linear"), fit(), predict(), decision_function(), roc_curve() and auc(). The boundary and the margin are given by:

$$w \cdot x + b = 0 \qquad Margin = 2 / ||w||$$

The decision_function() gives the distance of every test day from the boundary, and this score is used to draw the ROC curve. The ROC curve plots the True Positive Rate against the False Positive Rate for all possible thresholds:

$$TPR = TP / (TP + FN) \qquad FPR = FP / (FP + TN)$$

The area under the ROC curve (AUC) is a single number between 0.5 (random guessing) and 1 (perfect model).

iv. Analysis Results

The SVM gave an accuracy of 93.10%, the highest precision of 93.39%, a recall of 78.96% and an F1-score of 85.57%. It made only 72 false alerts, which is the lowest among all models, but it missed 271 alert days. The AUC of the ROC curve is 0.97, which means that the model separates the two classes very well when the threshold is changed.

v. Visualization

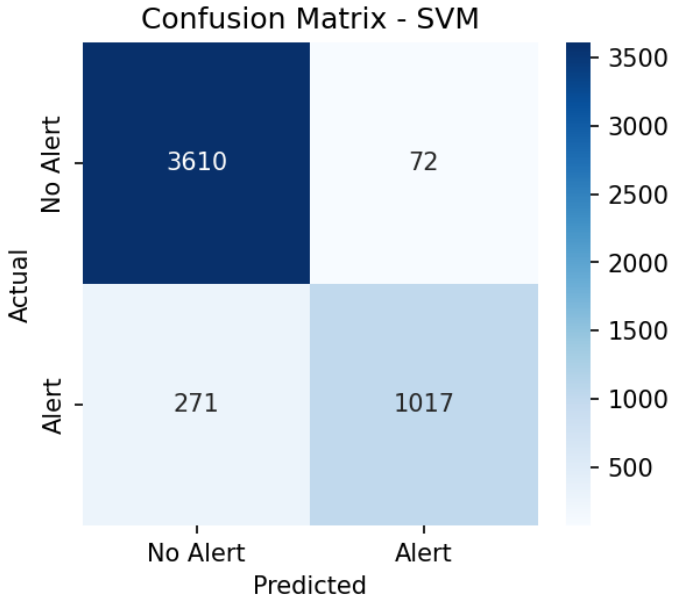


Figure 17: Confusion matrix of the SVM

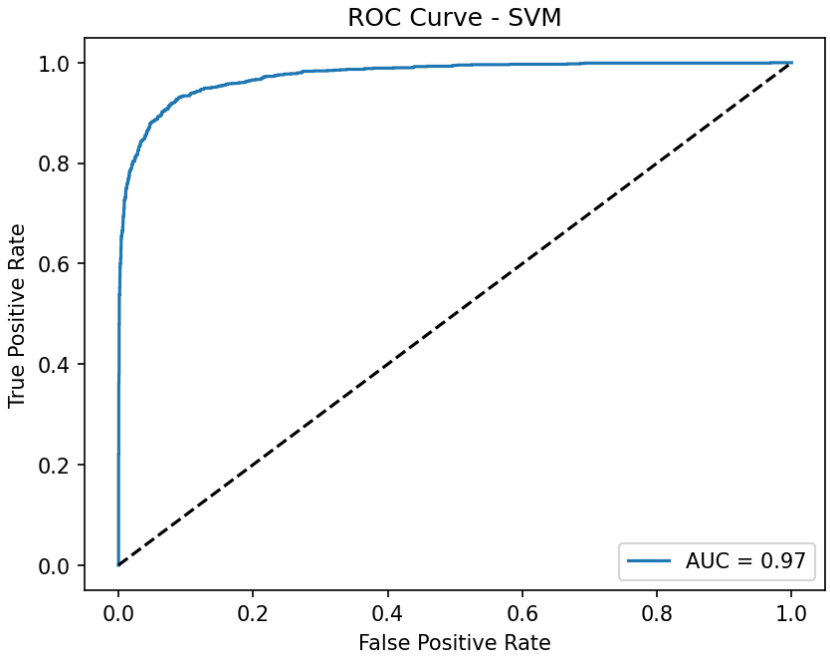


Figure 18: ROC curve of the SVM model

4.8 Comparison of the Classification Models

i. Introduction

After building the five classification models, they were compared with each other on the same test data. A fair comparison is possible because the same training rows, the same test rows and the same normalized inputs were used for all of them.

ii. General Description

Four metrics were used. Accuracy shows how many of all days were classified correctly. Precision shows how many of the predicted alert days were really alert days. Recall shows how many of the real alert days were found. F1-score combines precision and recall into one number. For an alert system recall is especially important, because a missed alert day means that people are not warned when the air is harmful. At the same time a very low precision would cause too many false alarms, so both must be considered.

iii. Specific Requirements, Functions and Formulas

The metrics were calculated with accuracy_score(), precision_score(), recall_score() and f1_score(), and the results of all models were collected with a for loop in one pandas DataFrame, which was sorted by recall. The formulas are the same as those given in Section 4.3.

iv. Analysis Results

Table 11: Comparison of the classification models (test data, sorted by recall)

Model	Accuracy	Precision	Recall	F1-score
KNN	93.84%	90.11%	85.64%	87.82%
Decision Tree	94.02%	91.67%	84.63%	88.01%
SVM	93.10%	93.39%	78.96%	85.57%
Naive Bayes	90.18%	82.73%	78.49%	80.56%
Logistic Regression	92.41%	92.77%	76.71%	83.98%

Table 12: Confusion matrix values of all classification models

Model	TN	FP	FN	TP
Logistic Regression	3605	77	300	988
KNN	3561	121	185	1103
Naive Bayes	3471	211	277	1011
Decision Tree	3583	99	198	1090
SVM	3610	72	271	1017

The main observations are as follows. The Decision Tree has the best accuracy (94.02%) and the best F1-score (88.01%). KNN has the best recall (85.64%), so it catches the largest number of alert days. SVM has the best precision (93.39%) and the fewest false alarms, but its recall is lower. Logistic Regression has a good accuracy but the lowest recall (76.71%). Gaussian Naive

Bayes is the weakest overall, because its independence assumption does not match the strongly correlated pollutants. If the aim is to miss as few alert days as possible, KNN is the best choice. If an easily explainable model with a very high overall score is needed, the Decision Tree is the best choice.

v. Visualization

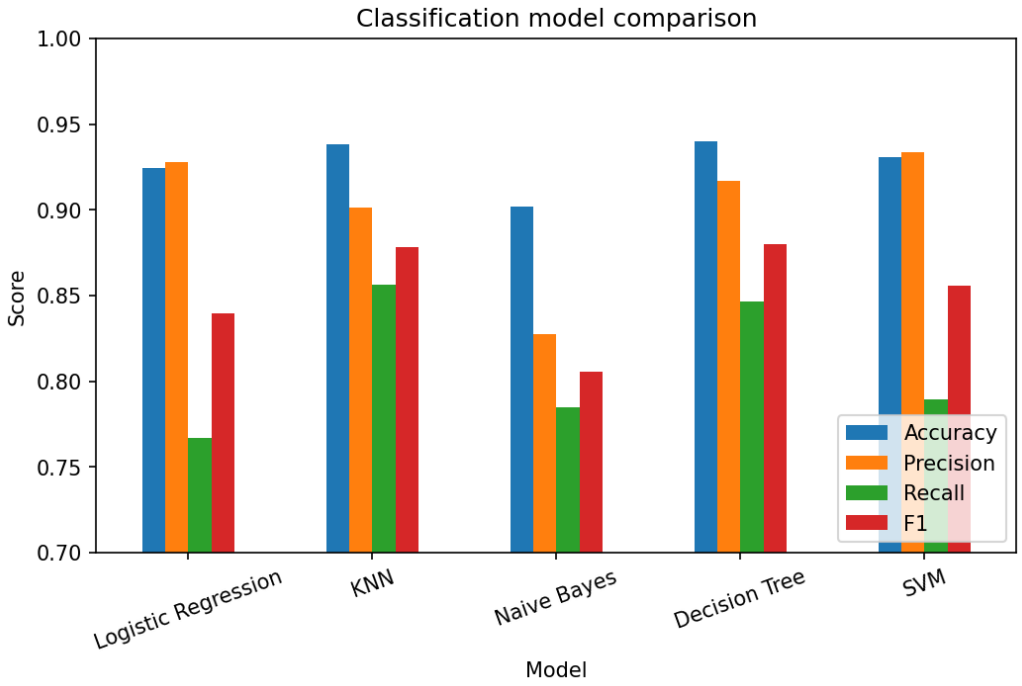


Figure 19: Comparison of the classification models

5. Conclusion

In this project the daily air quality data of 26 Indian cities from 2015 to 2020 was studied with the help of the Python techniques taught in the INT234 Predictive Analytics course. After removing the rows with a missing AQI and the column Xylene, 24,850 rows remained for the analysis, and the remaining missing pollutant values were filled with the column mean. The main conclusions of the project are given below.

- Air quality in India is a serious problem: about 25.98 percent of the recorded city-days had Poor, Very Poor or Severe air quality. Ahmedabad, Delhi, Patna, Gurugram and Lucknow had the highest average AQI, while Aizawl, Shillong and Coimbatore had the lowest.
- The AQI is strongly related to CO, PM2.5, NO2, PM10 and SO2. The air quality is clearly worse in the winter months (November to January) and best in the monsoon months (July and August).
- For predicting the AQI, Multiple Linear Regression ($R^2 = 0.8091$) is much better than Simple Linear Regression with PM2.5 alone ($R^2 = 0.4180$), because the AQI depends on many pollutants together.
- For predicting alert days, all five classifiers reached an accuracy of more than 90 percent. The Decision Tree gave the best accuracy (94.02%) and F1-score, KNN gave the best recall (85.64%) and SVM gave the best precision (93.39%).
- A very small Decision Tree that uses only PM2.5, CO and NO is almost as good as the other models. This shows that the alert days can be identified with a few important pollutants.

In short, the project shows that simple machine learning methods are able to estimate the AQI and to give a reliable warning about harmful-air days from pollutant readings. The results are, however, based on historical data and should be treated as a learning study and not as an official air quality warning.

6. Future Scope

The project can be improved and extended in the following ways.

- **Better handling of the data:** In this project the Min-Max scaling was applied to the whole data before the split, and the best K of KNN was chosen on the test data. In a stricter setup the scaler should be fitted only on the training data and K should be chosen on a separate validation set. Outliers could also be treated before building the regression models.
- **More features:** The city name, the year, weather data (temperature, humidity, wind speed and rainfall) and festival or crop-burning periods can be added as inputs to improve the prediction.
- **More models:** Techniques such as Random Forest, Gradient Boosting and neural networks can be tried, together with hyper-parameter tuning and cross-validation.
- **Time-series forecasting:** The present models estimate the AQI of the same day. Forecasting models can use the last few days to predict the AQI of the next days and give an advance warning.
- **Live data and application:** The model can be connected to live sensor data or to an open air quality service, and a web or mobile application can be built that sends alerts to people, schools and hospitals.
- **Class imbalance:** Methods such as class weights or resampling can be used to improve the recall of the alert class.

7. References

[1] Central Pollution Control Board (CPCB), Ministry of Environment, Forest and Climate Change, Government of India. National Air Quality Index. https://cpcb.nic.in

[2] Rao, R. Air Quality Data in India (2015 - 2020), dataset file city_day.csv. Kaggle. https://www.kaggle.com/datasets/rohanrao/air-quality-data-in-india

[3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. https://scikit-learn.org

[4] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 56-61.

[5] Harris, C. R. et al. (2020). Array programming with NumPy. Nature, 585, 357-362.

[6] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90-95.

[7] Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

[8] World Health Organization (2021). WHO global air quality guidelines: particulate matter (PM2.5 and PM10), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide. Geneva.

[9] Lecture notes and practical files of INT234 Predictive Analytics, Lovely Professional University, Phagwara, Punjab.